שיטות אופטימיזציה

פתרון בעיית

support vectors machine (SVM)

כבעיה פרימלית

מרצה יורם סגל

שקפים מבוססים על Quadratic Programming problems (QP)

Inner Product as a Decision Rule

The **Inner Product (Dot product)** is defined by the formula:

1 $\langle x, w \rangle = x_1 w_1 + \cdots + x_n w_n \;\; where \; x, w \in \mathbb{R}^n$

$x = \{a, b\}, w = \{1,0\}$

$\langle x, w \rangle = x_1 w_1 + x_2 w_2$

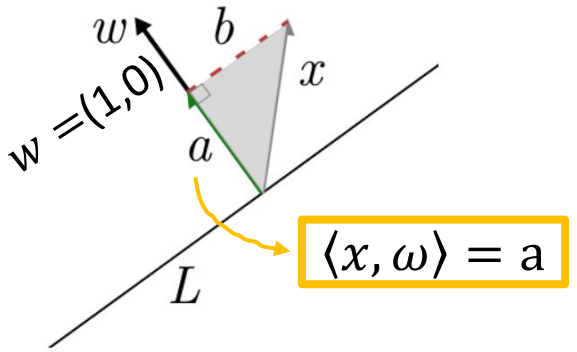
$x_1 = a \quad w_1 = 1$

$x_2 = b \quad w_2 = 0$

$\langle x, w \rangle = a \cdot 1 + b \cdot 0 = a$

w being a unit vector perpendicular to L ("the normal" to the line)

2 'a' is a scalar distance
(the distance from x to the plane L)



If we call a the (vector) leg of the triangle parallel to w, while b is the dotted line (as a vector, parallel to L), then as vectors $x = a + b$. The projection of x onto w is just a.

To define L, start by choosing a random vector X. Then, construct a system of axes containing W and L, ensuring that W is perpendicular to L.
It is important to note that the vector X is fixed and does not move. If you change the direction of W, the position of L changes, but X remains the same. Essentially, while you can build any system of axes around X, the direction of W is what determines the orientation of the entire system.

Yoram segal

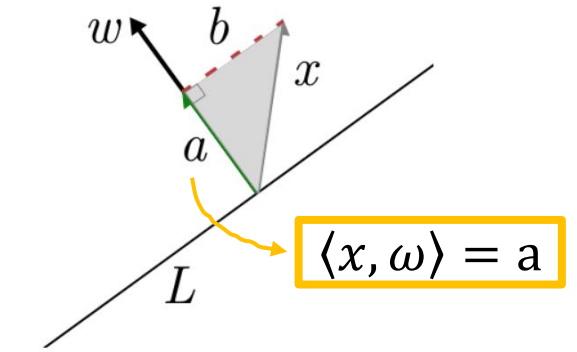
Inner Product

If you take any vector x, then the dot product $\langle x, w \rangle$ is positive if x is on the same side of L as w, and negative otherwise. The dot product is zero if and only if x is exactly on the line L, including when x is the zero vector.

Inner Product as a Decision Rule

'a' is a scalar distance
(the distance from x to the plane L)

2

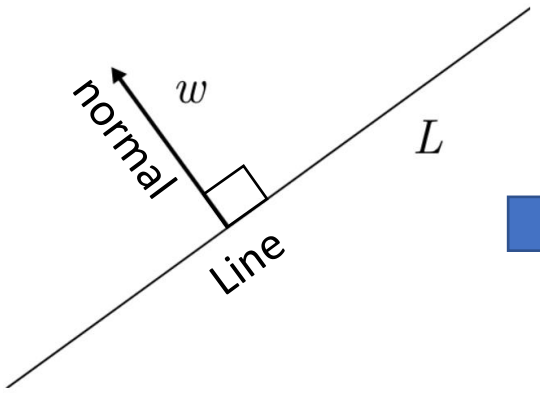


If we call a the (vector) leg of the triangle parallel to w, while b is the dotted line (as a vector, parallel to L), then as vectors $x = a + b$. The projection of x onto w is just a.

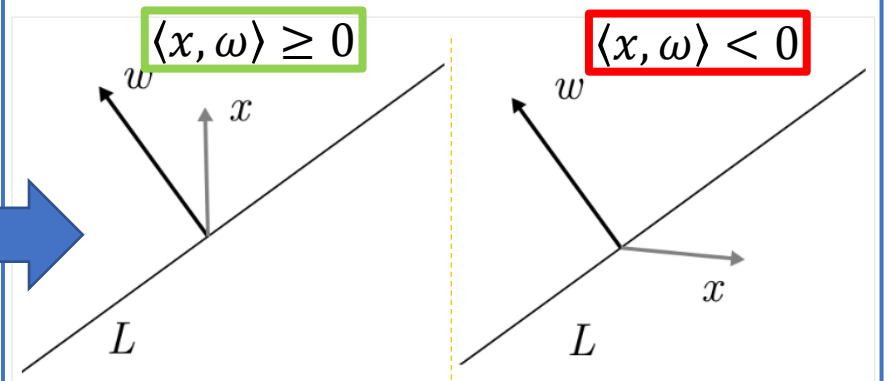
The **Inner Product (Dot product)** is defined by the formula:

1 $$\langle x, w \rangle = x_1 w_1 + \cdots + x_n w_n \ \ where\ x, w \in \mathbb{R}^n$$

Say we're in the Euclidean plane, and we have a line L passing through the origin, with w being a unit vector perpendicular to L ("the normal" to the line).



3

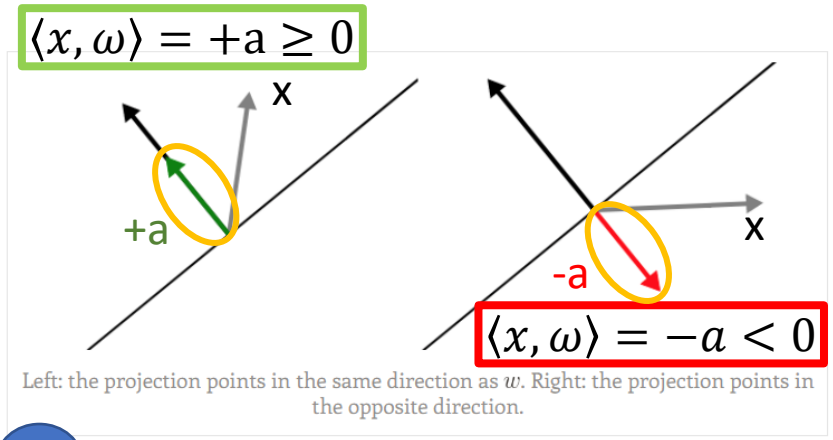


Left: the dot product of w and x is positive, meaning they are on the same side of w. Right: The dot product is negative, and they are on opposite sides.

$\langle x, w \rangle$ is positive if x is on the same side of L as w and negative otherwise.

4

$$a \in \mathbb{R}^1$$



Left: the projection points in the same direction as w. Right: the projection points in the opposite direction.

5

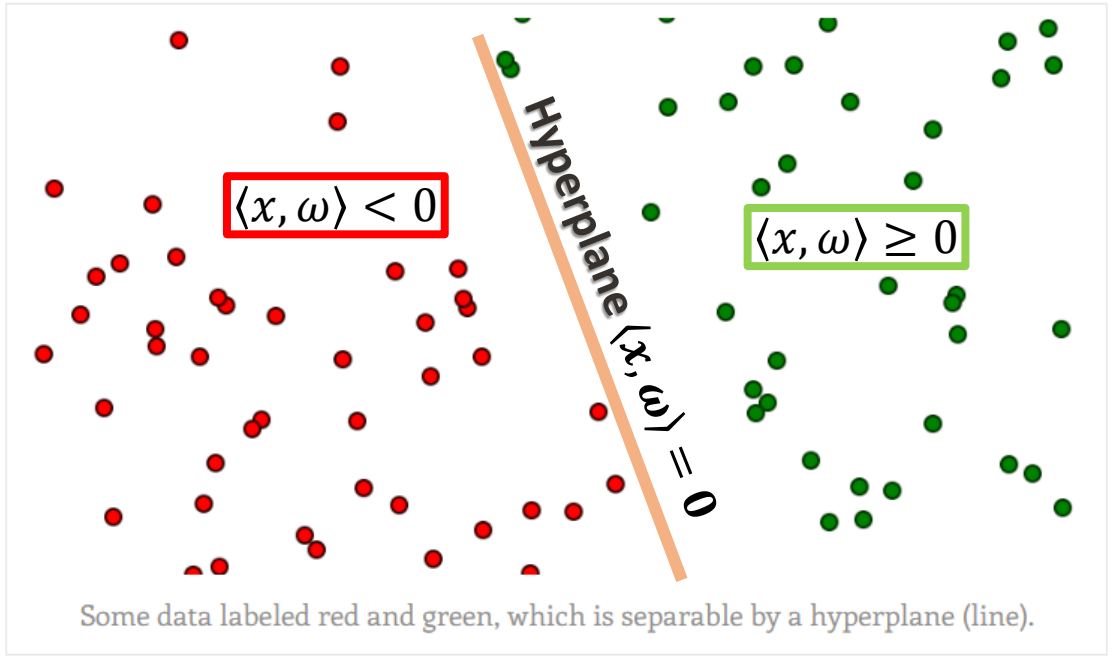
Yoram segal

Formulating the Support Vector Machine Optimization Problem via **Hyperplane**

w being a unit vector perpendicular to L ("the normal" to the line).

$$x \in \mathbb{R}^n$$

$$y \in \{1, -1\}$$



Some data labeled red and green, which is separable by a hyperplane (line).

$$\langle x, \omega \rangle = 0$$

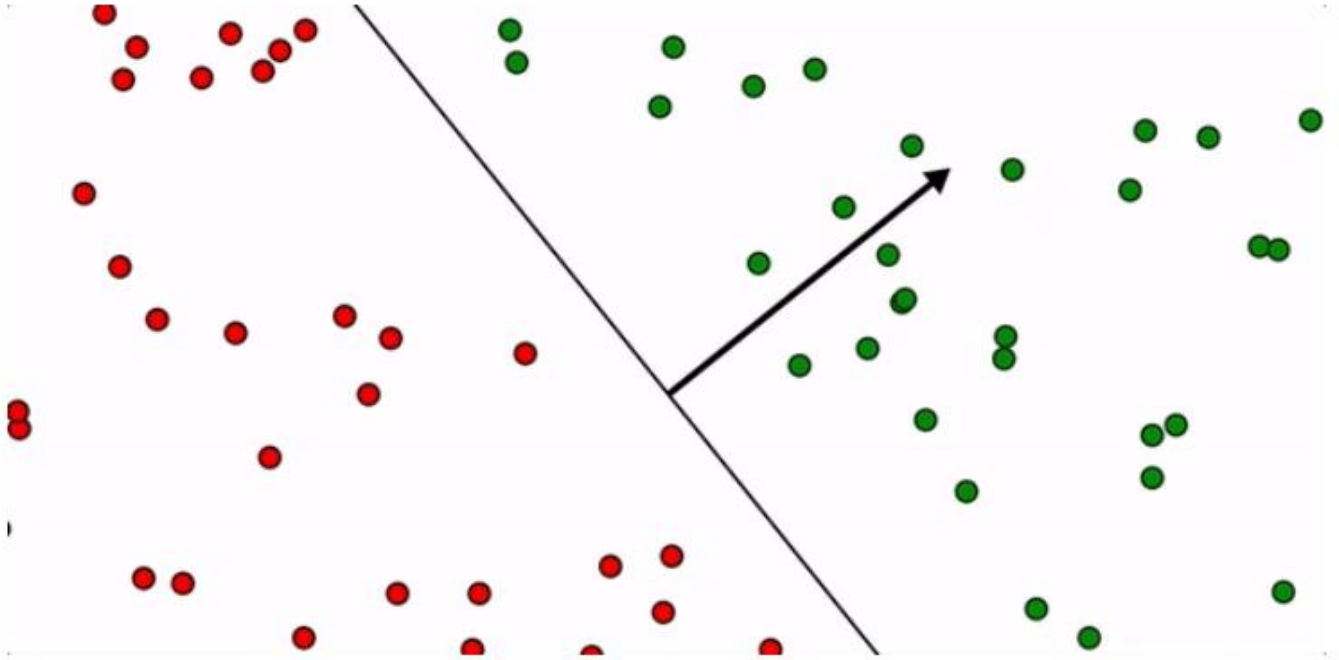
$$\langle x, w \rangle = 0 \cdot 1 + b \cdot 0 = 0$$

Hyperplane is defined by the solutions to the equation $\langle x, w \rangle = 0$

$$h_w(z) = \text{sign}(\langle w, x \rangle) \quad \Rightarrow \quad h_w(z) = \text{sign}(\langle w, x \rangle + b)$$

There are *many* possible separating hyperplanes for a given set of labeled training data

w being a unit vector perpendicular to L ("the normal" to the line).



קורדינאטות
Predictors $x \in \mathbb{R}^n$

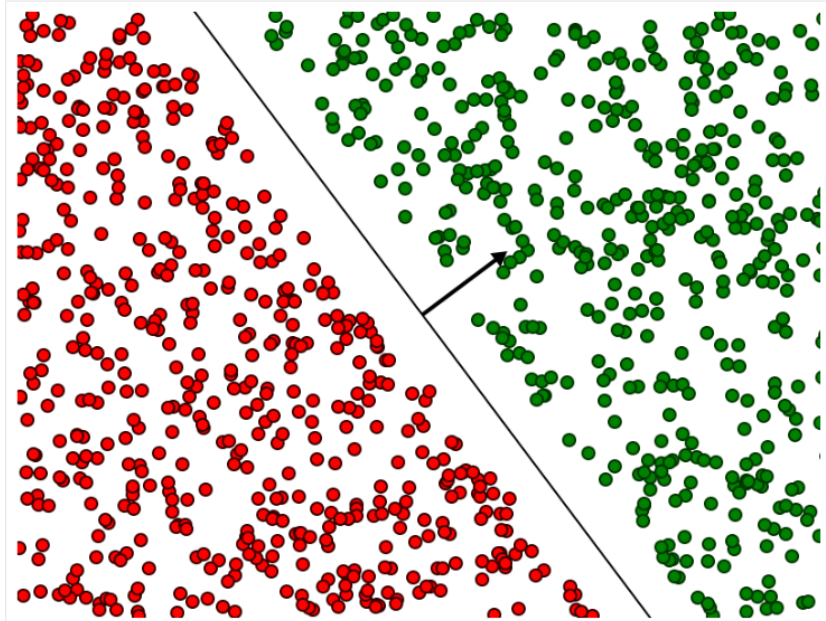
תיוג $y \in \{1, -1\}$
Response

Hyperplane is defined by the solutions to the equation $\langle x, w \rangle = 0$

$$h_w(z) = \text{sign}(\langle w, x \rangle)$$

For given set of points D such as:

$$D = \{(x_i, y_i) \mid i = 1, \ldots, m, x_i \in \mathbb{R}^n, y_i \in \{1, -1\}\}$$



While contrived, it's easy to see that the separating hyperplane is as far as possible from any training point.

The assumption of the SVM is that a hyperplane which separates the points, but is also *as far away from any training point as possible*, will generalize best.

Definition: The *geometric margin* of a hyperplane w with respect to a dataset D is the shortest distance from a training point x_i to the hyperplane defined by w.

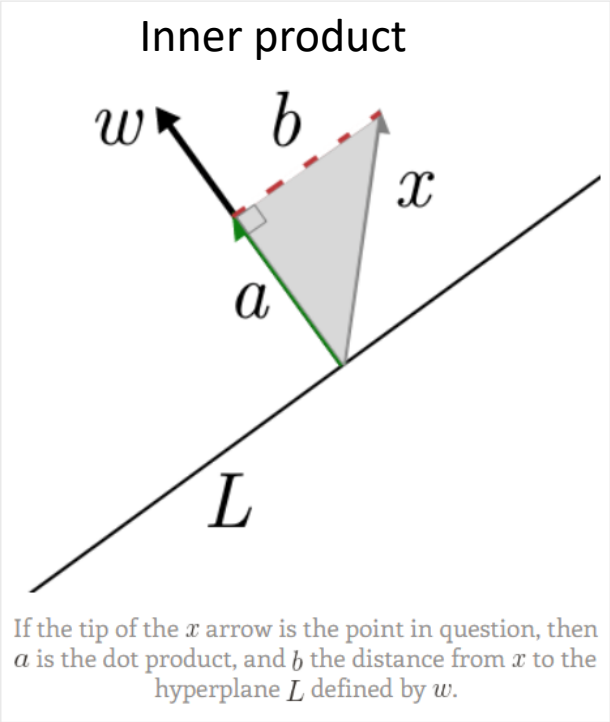
כלומר מחפשים את הנקודה הכי קרובה למישור ההפרדה w

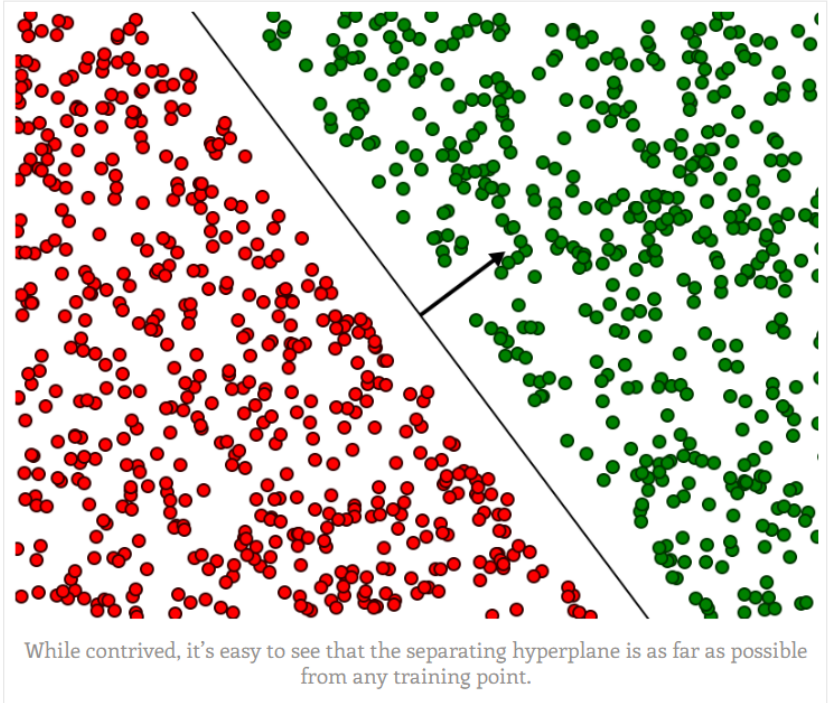
The best hyperplane has the largest possible margin.

אזור ההפרדה (התעלה) היא המרחק בין מישור ההפרדה לנקודה הכי קרובה כפול שניים

Inner product

Classification





The distance from x to the hyperplane defined by w is the same as the length of the projection of x onto w.
And this is just computed by an inner product.

מכפלה פנימית של כל דגימה (אדומה וירוקה) בווקטור w תיתן לנו את היטל הנקודה לכיוון w כלומר מרחק הנקודה ממישור w

A naive optimization objective

אנחנו מחפשים מישור הפרדה w הטוב ביותר.
כלומר נריץ את כל המישורים האפשריים.
עבור כל מישור נחפש את הנקודה הקרובה ביותר למישור (מבחן המינימום).
מתוך כל הנקודות שקיבלנו במבחן המינימום, נחפש את זאת שהמרחק שלה מהמישור W הוא הגדול ביותר (מבחן מקסימום)

$$\max_{w} \min_{x_i} \left| \left\langle x_i, \frac{w}{\|w\|} \right\rangle + b \right|$$

מוסיפים את b כי אולי מישור ההפרדה לא עובר דרך הראשית

$$\text{subject to} \quad \text{sign}(\langle x_i, w \rangle + b) = \text{sign}(y_i) \quad \text{for every } i = 1, \ldots, m$$

באילוץ, אנחנו דורשים כי הסימן של המכפלה הפנימית ושל תיוג הקבוצה יהיה זהים

The formulation is hard. The reason is it's horrifyingly nonlinear. In more detail:

a. The constraints are nonlinear due to the sign comparisons.
b. There's a min *and* a max! A priori, we have to do this because we don't know which point is going to be the closest to the hyperplane.
c. The objective is nonlinear in two ways: the absolute value and the projection requires you to take a norm and divide.

Trick 1: linearizing the constraints

$$\text{sign}(\langle x_i, w\rangle + b) = \text{sign}(y_i) \quad \Rightarrow \quad (\langle x_i, w\rangle + b) \cdot y_i \geq 0$$

Trick 1.5:
The optimal solution is midway between classes

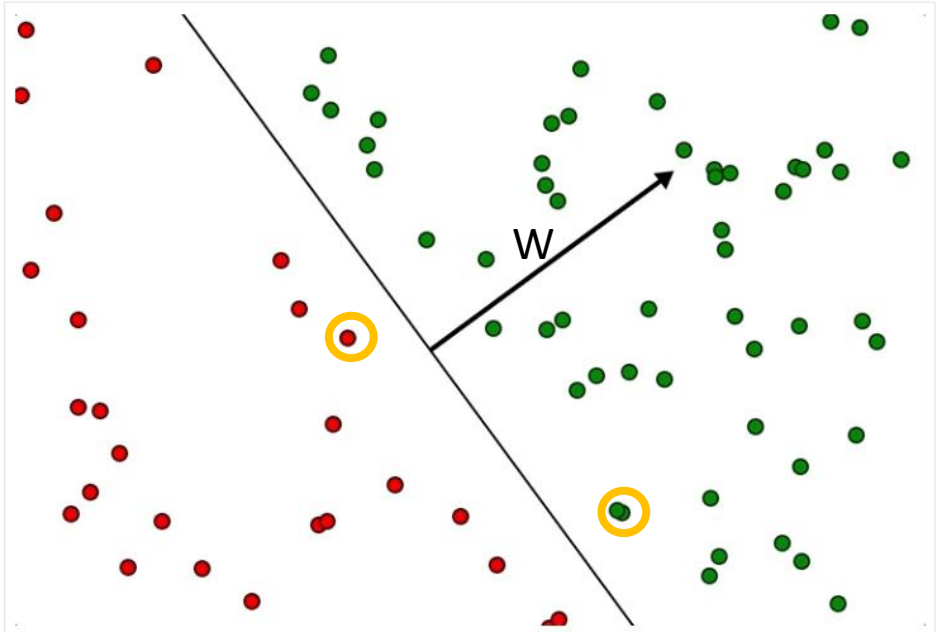
if W is the supposed optimal separating hyperplane, i.e. its margin is maximized, then it must necessarily be exactly halfway in between the closest points in the positive and negative classes.

$$\langle x_+, w\rangle + b = -(\langle x_-, w\rangle + b)$$

x_+ and x_- are the closest points in the positive and negative classes,

Trick 2: getting rid of the max + min

Increase or decrease the length of W without changing the decision boundary.



$$\langle a, w \rangle = ||w|| \cdot \langle a, w/||w|| \rangle.$$

Conclusion

בשיטה זו מחשבים את המרחק של הנקודה הקיצונית של הירוקים מקו ההפרדה
(את המרחק של הנקודה הירוקה שהכי קרובה לקו ההפרדה). מרחק זה מסומן כ||W||.
בנוסף, נחשב לכל הנקודות הירוקות את המרחק לקו ההפרדה.
(ע"י ביצוע מכפלה סקלרית בין ווקטור של כל נקודה עם ווקטור הנורמל למישור – ווקטור)W.
ברור כי המרחק של כל הנקודות הירוקות האחרות גדול או שווה למרחק ||W||.
נחלק את כל המרחקים של הנקודות הירוקות ב||W|| ונקבל כי המרחקים שלהם יהיו גדולים שווים לאחד.

נחזור על כל התהליך עבור הנקודות האדומות רק שהפעם נחשב ||W|| של הנקודה האדומה הקרובה ביותר.

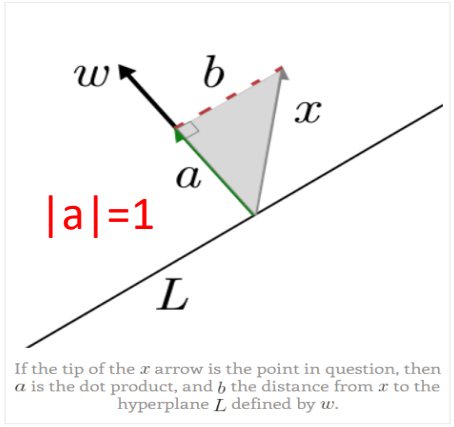
כלומר נקבל רוחב תעלה של פלוס מינוס אחד סביב הקו המפריד (רוחב 2)

הנקודות הקרובות ביותר נקראות ווקטורי תמיכה (support vectors).

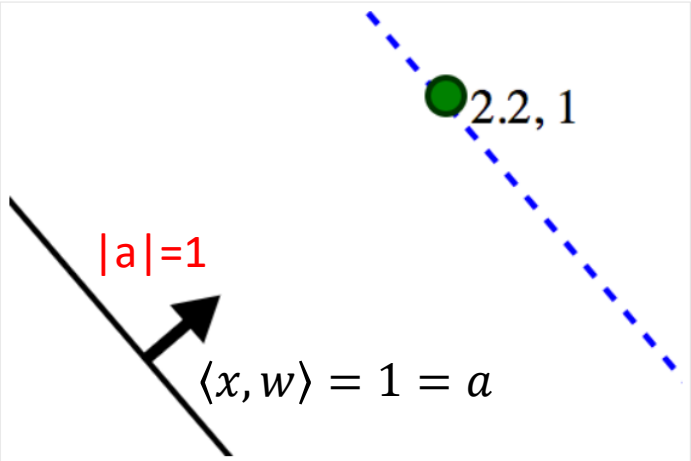
Trick 2: getting rid of the max + min (Cont. 1)

$$\langle a, w \rangle = \|w\| \cdot \langle a, w/\|w\| \rangle.$$

Now suppose we had the optimal hyperplane and its normal w. No matter how near (or far) the nearest positively labeled training point x is, we could scale the length of w to force $\langle x, w \rangle = 1$. This is the core of the trick. One consequence is that the *actual* distance from x to the hyperplane is $\frac{1}{\|w\|} = \langle x, w/\|w\| \rangle$.



If the tip of the x arrow is the point in question, then a is the dot product, and b the distance from x to the hyperplane L defined by w.



The same as above, but with the roles reversed. We're forcing the inner product of the point with w to be 1. The true distance is unchanged.

מנרמלים את הנקודה הכי קרובה
כך שהמרחק שלה למישור
ההפרדה יהיה 1

Trick 2: getting rid of the max + min (Cont. 2)

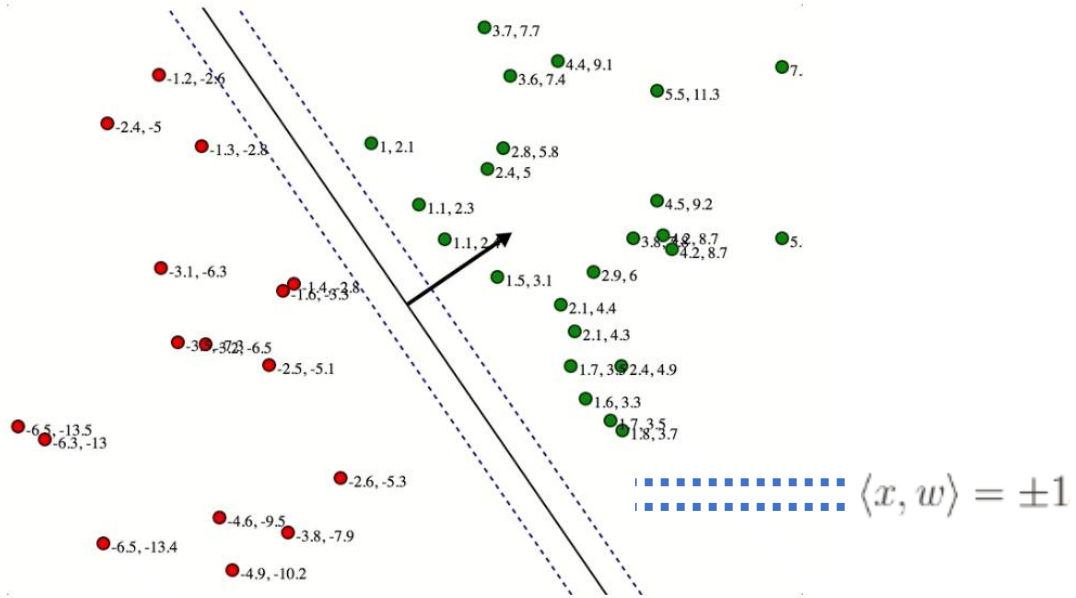
If we force the closest point to have inner product 1, then all other points will have inner product at least 1.

1. Our constraints change to $\langle x_i, w \rangle \cdot y_i \geq 1$ instead of ≥ 0

2. We no longer need to ask which point is closest to the candidate hyperplane!
   Because after all, we never cared *which* point it was, just *how far away* that closest point was.

3. Now we know that it's *exactly* $1/\|w\|$ away

In other words,
Provided all the constraints are satisfied,
The **optimization objective** is just to maximize $1/\|w\|$
A.k.a. to minimize $\|w\|$



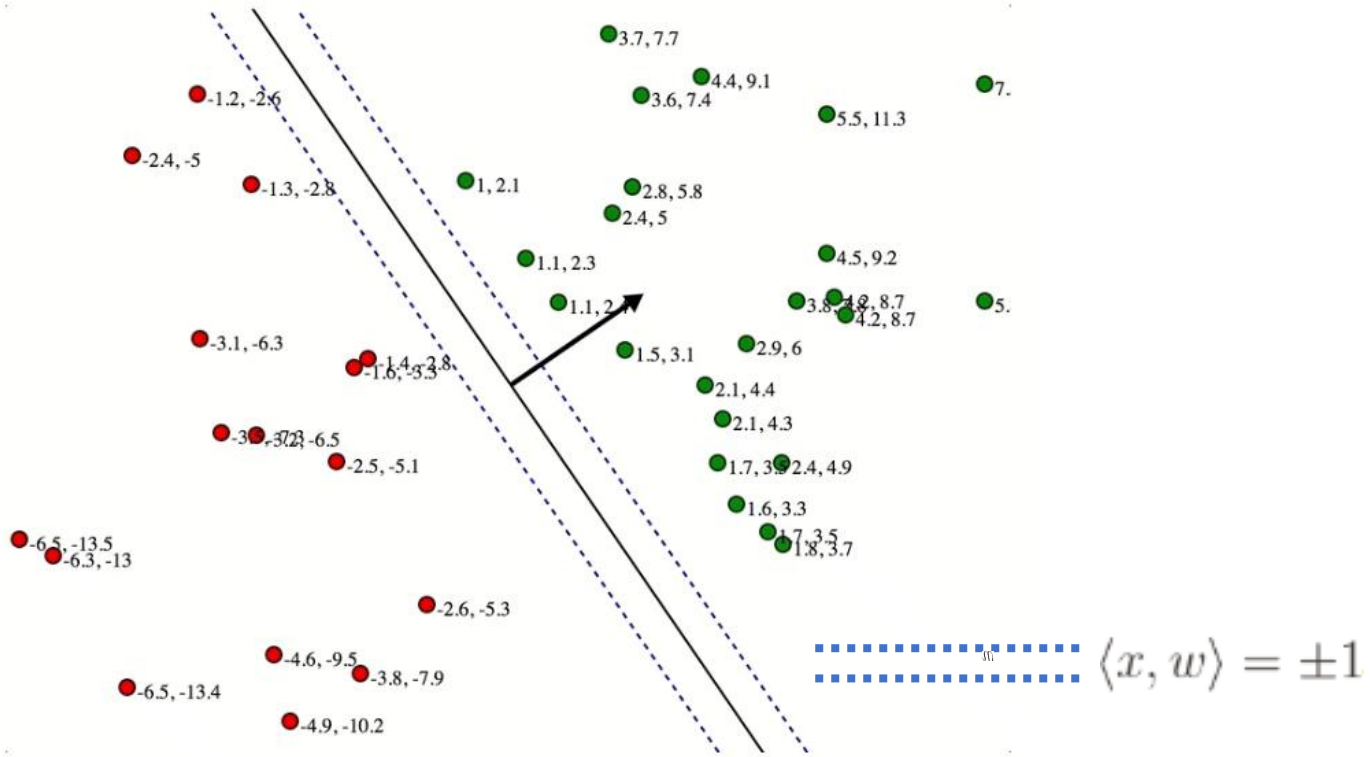
The true distance from that point to the candidate hyperplane

3.7, 6.6

The inner product with W

SVM Solution

In other words,
Provided all the constraints are satisfied,
The **optimization objective** is just to maximize $1/\|w\|$
A.k.a. to minimize $\|w\|$



To solve the SVM by hand,
Ensure the second number are :

- at least **+1** for all **green** points,
- at most **-1** for all **red** points,
- make **W** as short as possible.
  Shrinking **W** moves the blue lines farther away from the separator

The true distance from that point to the candidate hyperplane

3.7, 6.6

The inner product with W

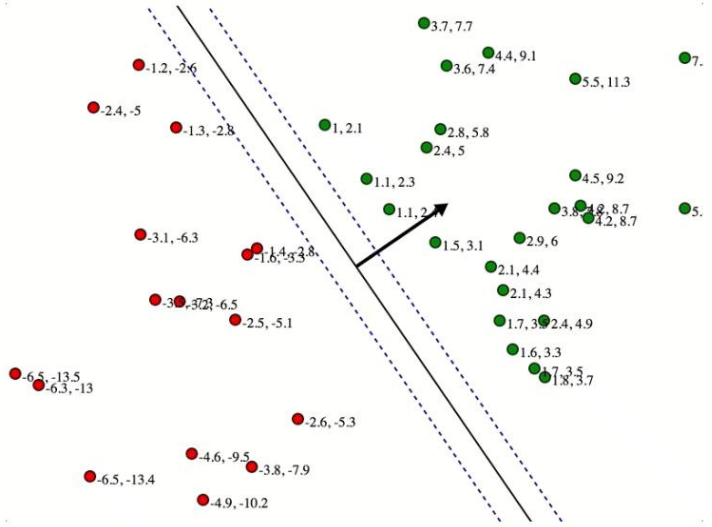
SVM Optimal Solution (by hand)

To solve the SVM by hand,
Ensure the second number are :

- at least **+1** for all **green** points,
- at most **-1** for all **red** points,
- make **W** as short as possible.
  Shrinking **W** moves the blue lines farther away from the separator

In order to satisfy the constraints the **blue lines** can't go further than any training point.

The optimum will have those **blue lines** touching a training point on each side.



$$\langle x, w \rangle = \pm 1$$

The true distance from that point to the candidate hyperplane

3.7, 6.6

The inner product with W

The final form of the SVM problem

The final note is that, since we are now minimizing $\|w\|$, a formula which includes a square root, we may as well minimize its square $\|w\|^2 = \sum_j w_j^2$. We will also multiply the objective by $1/2$, because when we eventually analyze this problem we will take a derivative, and the square in the exponent and the $1/2$ will cancel.

Our optimization problem is now the following (including the bias again):

$$\min_w \frac{1}{2}\|w\|^2$$
$$\text{subject to} \quad (\langle x_i, w\rangle + b) \cdot y_i \geq 1 \quad \text{for every } i = 1, \ldots, m$$

m-total number of all samples from the two groups

The final form of the SVM problem

Our optimization problem is now the following (including the bias again):

$$\min_{w} \frac{1}{2}\|w\|^2$$

$$\text{subject to} \quad (\langle x_i, w\rangle + b) \cdot y_i \geq 1 \quad \text{for every } i = 1, \ldots, m$$

m-number of samples

- This is much simpler to analyze.
- The constraints are all linear inequalities (which, because of linear programming, we know are tractable to optimize).
- The objective to minimize, however, is a convex quadratic function of the input variables → a sum of squares of the inputs.

How to solve QP problems with Matlab?

The final form of the SVM problem

Our optimization problem is now the following (including the bias again):

$$\min_{w} \frac{1}{2}\|w\|^2$$
$$\text{subject to} \quad (\langle x_i, w\rangle + b) \cdot y_i \geq 1 \quad \text{for every } i = 1, \ldots, m$$

- Such problems are generally called quadratic programming problems (or QPs, for short).
- There are general methods to find solutions! However, they often suffer from numerical stability issues and have less-than-satisfactory runtime.
- Luckily, the form in which we've expressed the support vector machine problem is specific enough that we can analyze it directly, and find a way to solve it without appealing to general-purpose numerical solvers.

See Matlab quadratic programming function: **quadprog()**

quadprog()

Quadratic programming

Solver for quadratic objective functions with linear constraints.

quadprog finds a minimum for a problem specified by

$$\min_x \frac{1}{2} x^T H x + f^T x \text{ such that } \begin{cases} A \cdot x \leq b, \\ Aeq \cdot x = beq, \\ lb \leq x \leq ub. \end{cases}$$

H, A, and Aeq are matrices, and f, b, beq, lb, ub, and x are vectors.

Quadratic function in matrix form

If $q = a_1 x^2 + a_2 y^2 + a_3 z^2 + a_4 xy + a_5 xz + a_6 yz$

then q is called a quadratic form (in variables x,y,z). There is a q value (a scalar) at every point. (To a physicist, q is probably the energy of a system with ingredients x,y,z.)

The matrix for q is

$$A = \begin{bmatrix} a_1 & \frac{1}{2}a_4 & \frac{1}{2}a_5 \\ \frac{1}{2}a_4 & a_2 & \frac{1}{2}a_6 \\ \frac{1}{2}a_5 & \frac{1}{2}a_6 & a_3 \end{bmatrix}$$

It's the symmetric matrix A with this connection to q:

(1) $$q = \begin{bmatrix} x & y & z \end{bmatrix} A \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

(X1	X2)			X1	X2		
			X1	aX_1^2	bx_1x_2		X1
			X2	bx_1x_2	cX_2^2		X2

quadprog() - Example

Find the minimum of

$$f(x) = \frac{1}{2}x_1^2 + x_2^2 - x_1x_2 - 2x_1 - 6x_2$$

subject to the constraints

$$x_1 + x_2 \le 2$$
$$-x_1 + 2x_2 \le 2$$
$$2x_1 + x_2 \le 3.$$

In quadprog syntax, this problem is to minimize

$$f(x) = \frac{1}{2}x^T Hx + f^T x,$$

where

$$H = \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}$$

$$f = \begin{bmatrix} -2 \\ -6 \end{bmatrix},$$

subject to the linear constraints.

$$\begin{bmatrix} a_1 & \frac{1}{2}a_4 & \frac{1}{2}a_5 \\ \frac{1}{2}a_4 & a_2 & \frac{1}{2}a_6 \\ \frac{1}{2}a_5 & \frac{1}{2}a_6 & a_3 \end{bmatrix}$$

$$a_1 x^2 + a_2 y^2 + a_3 z^2 + a_4 xy + a_5 xz + a_6 yz$$

quadprog() - SVM

$$\min_w \frac{1}{2}\|w\|^2$$
$$\text{subject to} \quad (\langle x_i, w\rangle + b) \cdot y_i \geq 1 \quad \text{for every } i = 1, \ldots, m$$

$$\min_w \frac{1}{2}\|w\|^2$$

$$H = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

לשם השימוש ב **quadprog()** מטריצה H הינה מטריצת יחידה ואילו הגורם הליניארי f הינו וקטור אפסים. quadprog מחייב להשתמש באילוצי אי שיוויון Ax <= b בעוד שאצלנו האי שיוויון הוא הפוך, לכן יש לכפול בקוד את הערכים של A ו b ב (-1) .

Support Vectors

The closest points lie on the two lines:

$$\langle x_i, w \rangle + b = \pm 1$$

The optimal hyperplane itself does not depend on *any* of the other points except these closest points:

$$\langle x_i, w \rangle + b = \pm 1$$

This fact is enough to give these closest points a special name: **the *support vectors***.

Support vectors “are all you need” with full rigor and detail, when we cast the optimization problem into the **“dual”** setting. The formulation described in this lesson called the **“primal”** problem.

Solve SVM duality problem via Support Vectors

Support vectors “are all you need” with full rigor and detail, when we cast the optimization problem into the **“dual”** setting. The formulation described in this lesson called the **“primal”** problem.

With the dual of the SVM problem, we will see explicitly that the hyperplane can be written as a linear combination of the support vectors.

As such, once we’ve found the optimal hyperplane, we can compress the training set into *just* the support vectors, and reproducing the same optimal solution becomes much, much faster.

We can also use the support vectors to augment the SVM to incorporate streaming data (throw out all non-support vectors after every retraining).